ΠΑΝΕΠΙΣΤΗΜΙΟ ΔΥΤΙΚΗΣ ΑΤΤΙΚΗΣ
ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΥΠΟΛΟΓΙΣΤΩΝ

ΣΧΕΔΙΑΣΗ ΨΗΦΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ
ΕΞΑΜΙΝΙΑΙΑ ΕΡΓΑΣΙΑ

ΤΣΑΚΙΡΗΣ ΜΙΧΑΗΛ
21390225

ΠΕΡΙΕΧΟΜΕΝΑ

Οδηγίες εκτέλεσης....3
Συστατικά μέρη & Σχηματική απεικόνιση....4
Περιγραφή κυκλώματος....5
ProgramCounter....5
ControlUnit....5
DataMemory....5
ANDgate....6
MUX32....6
MUX5....6
SignExtender....6
FullAdder....6
ALU....6
RegisterFile....7
ALUControlUnit....7
InstructionMemory....7
MIPS....7
Περιγραφή λειτουργίας κώδικα....8
Κώδικας....8
Testbench....18
Ενδεικτικό τρέξιμο....19

Οδηγίες εκτέλεσης

Στο παραδοτέο .zip πέρα από το παρόν .doc αρχείο, υπάρχουν δύο .vhd αρχεία που ονομάζονται MIPS.vhd και testbench όπου το ένα είναι ο ολοκληρωμένος κώδικας που υλοποιεί τον επεξεργαστή MIPS μαζί με τους κώδικες όλων των component σωστά συνδεδεμένα μεταξύ τους και το άλλο είναι το testbench του.

Βήματα:

> Simulate

> Start Simulation

> work

> work.testbench(testbench)

> Add

> To Wave

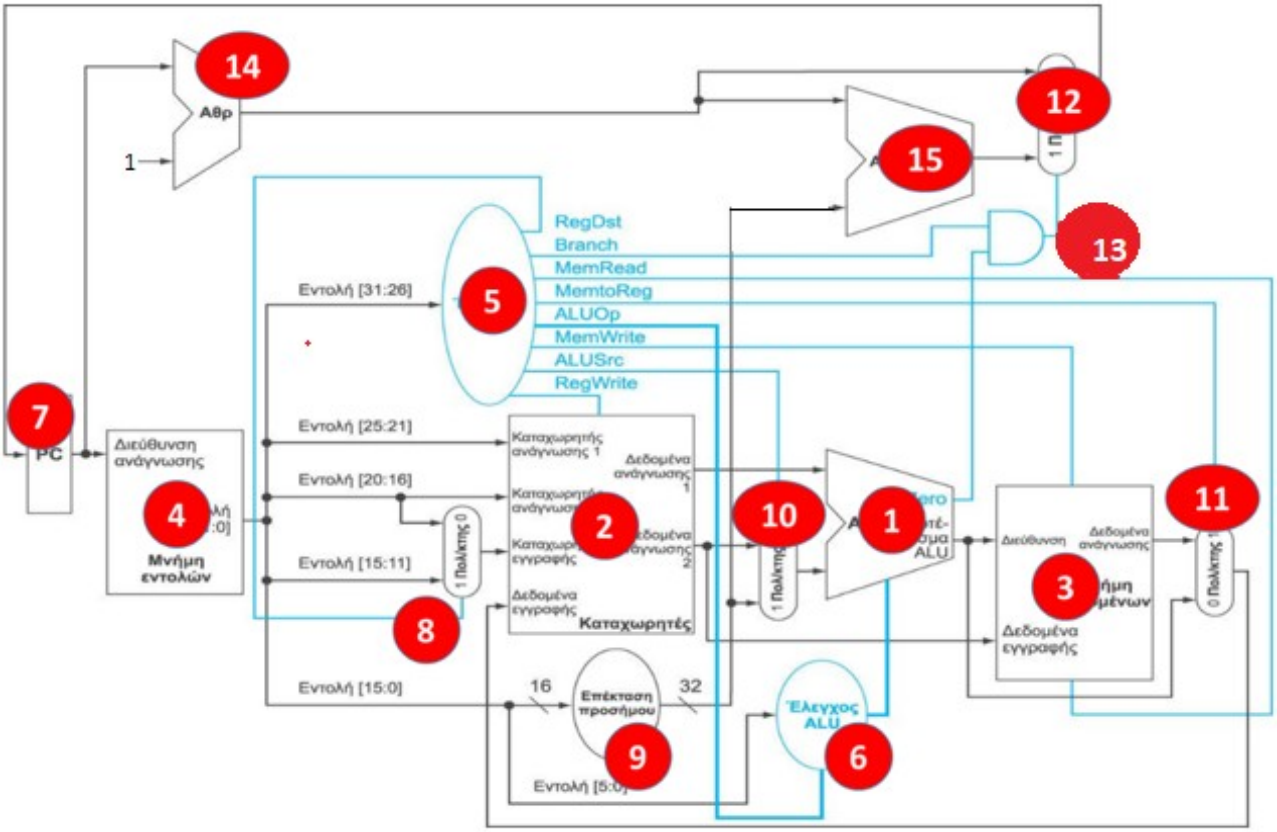
> All items in design

> Run (Simulate)

> Run -All

Συστατικά μέρη & Σχηματική απεικόνιση

1	ALU
2	Register file
3	Data Memory
4	Instruction Memory
5	Control Unit
6	ALU Control Unit
7	Program Counter
8	5bit MUX 2-to-1
9	Sign Extender
10, 11, 12	32bit MUX 2-to-1
13	AND gate with 2 inputs
14 ,15	32bit Full Adder



Περιγραφή κυκλώματος

Το κύκλωμα του κώδικα πραγματοποιεί έναν απλό (και ελαφρώς διαφοροποιημένο λόγω των προδιαγραφών της εργασίας) επεξεργαστή MIPS. Ειδικότερα, ο συγκεκριμένος επεξεργαστής αποτελείται από τον Μετρητή Προγράμματος (Program Counter), την Μνήμη Εντολών (Instruction Memory), τους Πολυπλέκτες MUX32 και MUX5, την Επέκταση Προσήμου (Sign Extender), την Μονάδα ελέγχου (Control Unit), την Αριθμητική Λογική Μονάδα (Arithmetic Logic Unit/ALU), την Μονάδα ελέγχου της ALU (ALU Control Unit), την Μνήμη Δεδομένων (Data Memory), τους Αθροιστές (Full Adder), μια λογική πύλη σύζευξης (AND gate) και το Αρχείο Καταχωρητών (Register File).

Καθένα από τα εξαρτήματα σχεδιάζεται ξεχωριστά και έχει το δικό του entity και architecture. Στο τέλος συνδυάζονται σε ένα ενιαίο architecture κάνοντας port τις εισόδους και τους εξόδους του κάθε εξαρτήματος κατάλληλα.

ProgramCounter

Ο μετρητής προγράμματος είναι ένας καταχωρητής που περιέχει την διεύθυνση της επόμενης εντολής (PCin) που θα εκτελεστεί. Κατά την εκτέλεση του προγράμματος, ο μετρητής αυξάνεται σειριακά κατά μια μονάδα σε κάθε θετική ακμή του ρολογιού (clock) για να δείξει στην επόμενη εντολή που πρέπει να εκτελεστεί. Επίσης, στην περίπτωση που δοθεί κατάλληλο σήμα (reset), ο μετρητής μηδενίζεται. Τέλος, η τιμή του μετρητή μπορεί να αλλάξει την ροή εκτέλεσης του προγράμματος στην περίπτωση εντολών τύπου branch (στην προκειμένη περίπτωση bne) και jump (που δεν πραγματεύονται στον συγκεκριμένο επεξεργαστή).

ControlUnit

Η μονάδα ελέγχου είναι υπεύθυνη για τον έλεγχο της εκτέλεσης των εντολών και τη διαχείριση των σημάτων ελέγχου εντολών. Αναγνωρίζει τον τύπο της κάθε εντολής και εκτελεί τις αντίστοιχες ενέργειες για να διαχειριστεί τα σήματα ελέγχου βάσει των 6 MSB της εντολής που εισάγεται (Opcode).

DataMemory

Η μνήμη δεδομένων αποτελεί τον χώρο αποθήκευσης δεδομένων του MIPS. Ειδικότερα, αποθηκεύει δεδομένα όπως ακέραιους αριθμούς και διευθύνσεις μνήμης και συχνά χρησιμοποιείται για λειτουργίες ανάγνωσης και εγγραφής. Στην προκειμένη περίπτωση αξιοποιείται ένα array (RAM) 16 θέσεων των 32 bit η κάθε μια.

ANDgate

Η πύλη AND είναι μια λογική πύλη που λαμβάνει 2 εισόδους (ANDin1, ANDin2) και παράγει ως αποτέλεσμα (ANDout) 0 ή 1. Η τιμή 1 παράγεται μόνο όταν και οι δύο είσοδοι έχουν τιμή 1, ενώ σε κάθε άλλη περίπτωση παράγεται η τιμή 0.

MUX32

Ο πολυπλέκτης 32-bit διαλέγει μεταξύ δύο εισόδων (MUXin1, MUXin2) βάσει της εισόδου ελέγχου (en) και επιστρέφει την αντίστοιχη έξοδο (MUXout). Αν η είσοδος ελέγχου (en) έχει τιμή 0, τότε η έξοδος είναι ίση με την πρώτη είσοδο (MUXin1), ενώ αν έχει τιμή 1, η έξοδος είναι ίση με τη δεύτερη (MUXin2).

MUX5

Ο πολυπλέκτης 5-bit διαλέγει μεταξύ δύο εισόδων (MUXin1, MUXin2) βάσει της εισόδου ελέγχου (en) και επιστρέφει την αντίστοιχη έξοδο (MUXout). Αν η είσοδος ελέγχου (en) έχει τιμή 0, τότε η έξοδος είναι ίση με την πρώτη είσοδο (MUXin1), ενώ αν έχει τιμή 1, η έξοδος είναι ίση με τη δεύτερη (MUXin2).

SignExtender

Η επέκταση προσήμου επεκτείνει το πλήθος των bit ενός σήματος απο 16 (SignIn) στα 32 (SignOut) βάσει του MSB. Πιο συγκεκριμένα, αν το MSB έχει τιμή 1 τότε προσθέτονται 16 άσσοι, αλλιώς αν έχει τιμή 0, προσθέτονται 16 μηδενικά.

FullAdder

Ο πλήρης αθροιστής πραγματοποιεί πρόσθεση δύο αριθμών των 32 bit (FAin1, FAin2) και παράγει ένα αποτέλεσμα των 32 bit (Sum). Γενικά, στο κύκλωμα ενός πλήρους αθροιστή υπάρχει πιθανότητα να υπάρχει ένα κρατούμενο από προηγούμενη πρόσθεση (Cin) ή η τρέχουσα πράξη να παράξει δικό της (Cout). Ωστόσο, στους επεξεργαστές MIPS γενικά δεν αξιοποιούνται τα κρατούμενα ψηφία.

ALU

Η Αριθμητική Λογική Μονάδα (ALU) εκτελεί αριθμητικές και λογικές λειτουργίες πάνω σε δεδομένα. Ειδικότερα, επεξεργάζεται δύο εισόδους (ALUin1, ALUin2) και έναν κωδικό ελέγχου (op) που καθορίζει την επιθυμητή λειτουργία και παράγει μια αντίστοιχη έξοδο (ALUout, Zero).

RegisterFile

Το αρχείο καταχωρητών είναι ένας χώρος φύλαξης καταχωρητών στους οποίους επιτρέπεται η ανάγνωση και η εγγραφή τιμών (RegOut1, RegOut2). Στην προκειμένη περίπτωση, τα δεδομένα διαχειρίζονται μέσω ενός array (reg) 16 θέσεων των 32 bit η κάθε μια.

ALUControlUnit

Η μονάδα ελέγχου της αριθμητικής λογικής μονάδας είναι ένας κωδικοποιητής που δέχεται μια είσοδο (funct) και έναν κωδικό ελέγχου (ALUop) και επιστρέφει μια αντίστοιχη έξοδο (ALUControlOut) η οποία καθορίζει τον τρόπο λειτουργίας της Αριθμητικής Λογικής Μονάδας (ALU).

InstructionMemory

Η μνήμη εντολών αποθηκεύει τις εντολές προς εκτέλεση και βάσει της εισόδου που λαμβάνει (IMin) προωθεί την κάθε εντολή στην αντίστοιχη διεύθυνση (IMout). Στην προκειμένη περίπτωση, οι εντολές προς εκτέλεση αποθηκεύονται σε ένα array (commands) 16 θέσεων των 32 bit η κάθε μια.

MIPS

Ο επεξεργαστής MIPS είναι αποτέλεσμα της σύνδεσης όλων των παραπάνω εξαρτημάτων. Η σύνδεση μεταξύ τους επιτυγχάνεται μέσω πολλαπλών σημάτων (signals) από και προς εξαρτήματα.

Περιγραφή λειτουργίας κώδικα

Ο συγκεκριμένος κώδικας εκτελεί διάφορες πράξεις υπολογισμού και πρόσβασης μνήμης. Ειδικότερα, εκτελεί πρόσθεση (add, addi), αποθήκευση δεδομένων στη μνήμη (sw) και διακλάδωση (bne). Ειδικότερα, ο κώδικας εκτελεί έναν βρόχο επανάληψης που αποθηκεύει την ακολουθία των αριθμών 1, 2, 3 στη μνήμη.

Κώδικας

```
--ProgramCounter
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity ProgramCounter is port(
          PCin: in std_logic_vector(31 downto 0);
          PCout: out std_logic_vector(31 downto 0);
          clock, reset: in std_logic);
end ProgramCounter;

architecture behavioral of ProgramCounter is
begin
          process (clock, reset) begin
                    if reset = '1' then
                              PCout <= (others => '0'); --resetting PC
                    elsif rising_edge(clock) then
                              PCout <= PCin; -- where PCin is PC+1
                    end if;
          end process;
end behavioral;



--ANDgate
library ieee;
use ieee.std_logic_1164.all;

entity ANDgate is port(
          ANDin1, ANDin2: in std_logic;
          ANDout: out std_logic);
end ANDgate;

architecture dataflow of ANDgate is
begin
          ANDout <= ANDin1 and ANDin2;
end dataflow;



--MUX32
library ieee;
use ieee.std_logic_1164.all;

entity MUX32 is port(
          MUXin1, MUXin2: in std_logic_vector(31 downto 0);
          MUXout: out std_logic_vector(31 downto 0);
          en: in std_logic);
```

```
end MUX32;

architecture dataflow of MUX32 is
begin
          MUXout <= MUXin1 when en = '0' else MUXin2;
end dataflow;



--MUX5
library ieee;
use ieee.std_logic_1164.all;

entity MUX5 is port(
          MUXin1, MUXin2: in std_logic_vector(4 downto 0);
          MUXout: out std_logic_vector(4 downto 0);
          en: in std_logic);
end MUX5;

architecture dataflow of MUX5 is
begin
          MUXout <= MUXin1 when en='0' else MUXin2;
end dataflow;



--FullAdder
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity FullAdder is port (
        FAin1, FAin2: in std_logic_vector(31 downto 0);
        Cin: in std_logic_vector(0 downto 0);--must be a vector or else there is an
error in conversion below
        Cout: out std_logic;
        Sum: out std_logic_vector(31 downto 0));
end FullAdder;

architecture dataflow of FullAdder is
    signal tmp: std_logic_vector(32 downto 0);
begin
    tmp <= std_logic_vector(to_signed( to_integer(signed(FAin1)) +
to_integer(signed(FAin2)) + to_integer(signed(Cin)), 33)); --TL;DR signed addition
    -- the inputs are converted to signed integer values and then the result is
reconverted 33-bit singed integer representation which is assigned as a logic vector
    Cout <= tmp(32);
    Sum <= tmp(31 downto 0);
end dataflow;



--SignExtender
library ieee;
use ieee.std_logic_1164.all;

entity SignExtender is port(
          SignIn: in std_logic_vector(15 downto 0);
          SignOut: out std_logic_vector(31 downto 0));
end SignExtender;

architecture dataflow of SignExtender is
```

```
          signal ones: std_logic_vector(15 downto 0) := (others => '1');
          signal zeros: std_logic_vector(15 downto 0) := (others => '0');
begin
          SignOut <= ones & SignIn when SignIn(15) = '1'  -- adding 16 '1' bits
                    else zeros & SignIn when SignIn(15) = '0';  -- adding 16 '0' bits
end dataflow;



--InstructionMemory
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity InstructionMemory is port(
          IMin: in std_logic_vector(31 downto 0);
          IMout: out std_logic_vector(31 downto 0));
end InstructionMemory;

architecture behavioral of InstructionMemory is
          type IM_array is array(0 to 15) of std_logic_vector(31 downto 0); -- 16 slots
of 32 bits each
          signal commands: IM_array:=(X"20000000", -- addi $0, $0, 0

          X"20420000", -- addi $2, $2, 0

          X"20820000", -- addi $2, $4, 0

          X"20030001", -- addi $3, $0, 1

          X"20050003", -- addi $5, $0, 3

          X"00603020", -- L1: add $6, $3, $0

          X"AC860000", -- sw $6, 0($4)

          X"20630001", -- addi $3, $3, 1

          X"20840001", -- addi $4, $4, 1

          X"20A5FFFF", -- addi $5, $5, -1

          X"14A0FFFA", -- bne $5,$0,L1

          X"00000000", -- slot padding

          X"00000000", -- slot padding

          X"00000000", -- slot padding

          X"00000000", -- slot padding

          X"00000000");-- slot padding

begin

          IMout <= commands(to_integer(unsigned(IMin))); --assigning a value
(instruction) from the commands array to the signal IMout based on the value of IMin

end behavioral;
```

```
--DataMemory
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity DataMemory is port(
          clock: in std_logic;
          DMin: in std_logic_vector(31 downto 0);
          WriteData:  in std_logic_vector(31 downto 0);
          MemWrite: in std_logic;
          MemRead: in std_logic;
          DMout: out std_logic_vector(31 downto 0);
          reset: in std_logic);
end entity;

architecture dataflow of DataMemory is
          type DM_array is array(0 to 15) of std_logic_vector(31 downto 0); --memory
storage of 16 elements that fit 32 bit each
          signal address: integer:= 0;
          signal RAM: DM_array := (others => (others => '0')); --initialization of
array RAM with 0
begin
                    address <= to_integer(unsigned(DMin)) when
(to_integer(unsigned(DMin)) <= 15) else 0;
                    RAM(address) <= WriteData when (MemWrite = '1' and reset = '0' and
rising_edge(clock));
                    DMout <= RAM(address) when (reset = '0' and MemRead = '1') else
x"00000000"; --DMout becomes the value stored in the address
end dataflow;



--ControlUnit
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity ControlUnit is port(
          clock: in std_logic;
          OPcode: in std_logic_vector(5 downto 0);
          RegWrite: out std_logic := '0';
          ALUsrc: out std_logic;
          ALUop: out std_logic_vector(2 downto 0);
          MemWrite: out std_logic;
          MemRead: out std_logic;
          RegDst: out std_logic;
          MemToReg: out std_logic;
          Branch: out std_logic);
end ControlUnit;

architecture dataflow of ControlUnit is
begin
          with OPcode select
                    RegWrite <=
                              ('1' and clock) when "100011", --load
                              ('1' and clock) when "000000", --arithmetic
                              ('1' and clock) when "001000", --addi
                              '0' when others;

          with OPcode select
                    ALUsrc <=
```

```
                                '1' after 2 ns when "100011", --load
                                '1' after 2 ns when "101011", --store
                                '1' after 2 ns when "001000", --addi
                                '0' when others;

          with OPcode select
                    ALUop <=
                                "000" after 2 ns when "000000", --arithmetic
                                "001" after 2 ns when "100011", --load
                                "001" after 2 ns when "101011", --store
                                "011" after 2 ns when "000101", --bne
                                "100" after 2 ns when "001000", --addi
                                "111" when others;

          with OPcode select
                    MemWrite <=
                                '1' after 10 ns when "101011", --store
                                '0' when others;

          with OPcode select
                    MemRead <=
                                '1' after 2 ns when "100011", --read
                                '0' when others;

          with OPcode select
                    MemToReg <=
                                '1' after 2 ns when "100011", --read
                                '0' when others;

          with OPcode select
                    RegDst <=
                                '0' when "100011", --load
                                '0' when "001000", --addi
                                '1' when others;

          with OPcode select
                    Branch <=
                                '1' when "000101", --bne
                                '0' when others;

end dataflow;



--ALUcontrolUnit
library ieee;
use ieee.std_logic_1164.all;

entity ALUcontrolUnit is port(
          ALUop: in std_logic_vector(2 downto 0);
          Funct: in std_logic_vector(5 downto 0);
          ALUcontrolUnitOut: out std_logic_vector(3 downto 0));
end ALUcontrolUnit;

architecture dataflow of ALUcontrolUnit is

          signal tmpFunct: std_logic_vector(3 downto 0) := (others => '0');
          signal tmpALUop: std_logic_vector(3 downto 0) := (others => '0');
begin
          with Funct select
                    tmpFunct <=
                                "0001" when "100000", --add/sub
```

```
                                "1111" when others;

                        with ALUop select
                                tmpALUop <=
                                        "0001" when "001", --store/load
                                        "1011" when "011", --bne
                                        "1101" when "100", --addi
                                        "1111" when others;

                        with ALUop select
                                ALUcontrolUnitOut <=
                                        tmpFunct when "000",
                                        tmpALUop when others;
end dataflow;



--ALU
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity ALU is port(
                        ALUin1, ALUin2: in std_logic_vector(31 downto 0);
                        op: in std_logic_vector(3 downto 0);
                        ALUout: out std_logic_vector(31 downto 0);
                        Zero: out std_logic);
end ALU;

architecture dataflow of ALU is

            component FullAdder port(
                        FAin1, FAin2: in std_logic_vector(31 downto 0);
                        Cin: in std_logic_vector(0 downto 0);
                        Cout: out std_logic;
                        Sum: out std_logic_vector(31 downto 0));
            end component;

            signal Cout: std_logic;
            signal FAout: std_logic_vector(31 downto 0);
            signal X: std_logic_vector(31 downto 0) := (others => 'X');

begin
            FA_ALU: FullAdder port map( FAin1 => ALUin1, FAin2 => ALUin2, Cin => "0", Sum
=> FAout, Cout => Cout);

            with op select
                        ALUout <=
                                FAout when "0001", --sum
                                FAout when "1101", --addi
                                X when others;

            Zero <= '1' when ( (ALUin1 /= ALUin2) and (op = "1011") ) else '0';

end dataflow;



--RegisterFile
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
```

```
entity RegisterFile is port (
                    clock: in std_logic;
        RegIn1: in std_logic_vector(4 downto 0);
        RegIn2: in std_logic_vector(4 downto 0);
        RegWriteIn: in std_logic_vector(4 downto 0);
        DataWriteIn: in std_logic_vector(31 downto 0);
        RegWrite: in std_logic;
        RegOut1: out std_logic_vector(31 downto 0);
        RegOut2: out std_logic_vector(31 downto 0));
end RegisterFile;

architecture behavioral of RegisterFile is
    type RF_ARRAY is array(0 to 15) of std_logic_vector(31 downto 0);

          signal X: std_logic_vector(31 downto 0) := (others => 'X');
    signal reg: RF_ARRAY := (others => (others => '0'));
          signal RegWriteDelayed: std_logic;
begin
          process (clock) begin
                    RegOut1 <= reg(to_integer(unsigned(RegIn1)));
                    RegOut2 <= reg(to_integer(unsigned(RegIn2)));
                    RegWriteDelayed <= transport RegWrite after 1 ns;
                    if (rising_edge(clock) and RegWriteDelayed = '1' and DataWriteIn /=
X) then
                              reg(to_integer(unsigned(RegWriteIn))) <= DataWriteIn;
                    end if;
          end process;
end behavioral;



--MIPS
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity MIPS is port(
          reset: in std_logic;
          clock: in std_logic);
end MIPS;

architecture structural of MIPS is

          component ALU port(
                    ALUin1, ALUin2: in std_logic_vector(31 downto 0);
                    op: in std_logic_vector(3 downto 0);
                    ALUout: out std_logic_vector(31 downto 0);
                    Zero: out std_logic);
          end component;

          component RegisterFile port(
                    clock: in std_logic;
                    RegIn1: in std_logic_vector(4 downto 0);
                    RegIn2: in std_logic_vector(4 downto 0);
                    RegWriteIn: in std_logic_vector(4 downto 0);
                    DataWriteIn: in std_logic_vector(31 downto 0);
                    RegWrite: in std_logic;
                    RegOut1: out std_logic_vector(31 downto 0);
                    RegOut2: out std_logic_vector(31 downto 0));
          end component;
```

```
component DataMemory port(
          clock: in std_logic;
          DMin: in std_logic_vector(31 downto 0);
          WriteData:  in std_logic_vector(31 downto 0);
          MemWrite: in std_logic;
          MemRead: in std_logic;
          DMout: out std_logic_vector(31 downto 0);
          reset: in std_logic);
end component;

component InstructionMemory port(
          IMin: in std_logic_vector(31 downto 0);
          IMout: out std_logic_vector(31 downto 0));
end component;

component ControlUnit port(
          clock: in std_logic;
          OPcode: in std_logic_vector(5 downto 0);
          RegWrite: out std_logic;
          ALUsrc: out std_logic;
          ALUop: out std_logic_vector(2 downto 0);
          MemWrite: out std_logic;
          MemRead: out std_logic;
          RegDst: out std_logic;
          MemToReg: out std_logic;
          Branch: out std_logic);
end component;

component ALUcontrolUnit port(
          ALUop: in std_logic_vector(2 downto 0);
          Funct: in std_logic_vector(5 downto 0);
          ALUcontrolUnitOut: out std_logic_vector(3 downto 0));
end component;

component ProgramCounter port(
          PCin: in std_logic_vector(31 downto 0);
          PCout: out std_logic_vector(31 downto 0);
          clock, reset: in std_logic);
end component;

component MUX5 port(
          MUXin1, MUXin2: in std_logic_vector(4 downto 0);
          MUXout: out std_logic_vector(4 downto 0);
          en: in std_logic);
end component;

component SignExtender port(
          SignIn: in std_logic_vector(15 downto 0);
          SignOut: out std_logic_vector(31 downto 0));
end component;

component MUX32 port(
          MUXin1, MUXin2: in std_logic_vector(31 downto 0);
          MUXout: out std_logic_vector(31 downto 0);
          en: in std_logic);
end component;

component ANDgate is port(
          ANDin1, ANDin2: in std_logic;
          ANDout: out std_logic);
end component;
```

```
          component FullAdder port(
                    FAin1, FAin2: in std_logic_vector(31 downto 0);
                    Cin: in std_logic_vector(0 downto 0);
                    Cout: out std_logic;
                    Sum: out std_logic_vector(31 downto 0));
          end component;

--Control Unit signals
signal RegDst: std_logic;
signal Branch: std_logic;
signal MemRead: std_logic;
signal MemToReg: std_logic;
signal ALUop: std_logic_vector(2 downto 0);
signal MemWrite: std_logic;
signal ALUSrc: std_logic;
signal RegWrite: std_logic;
signal Zero: std_logic;
signal BranchOut: std_logic; --output of Branch/AND gate

--Program Counter signals
signal PCtoFAorIM: std_logic_vector(31 downto 0); -- PC output towards FullAdder or
InstructionMemory
signal number1: std_logic_vector(31 downto 0); --integer number 1
signal FAplusplusOut: std_logic_vector(31 downto 0); -- output of FullAdder + 1
signal FA2toMUX: std_logic_vector(31 downto 0); --FA2 to MUX
signal MUXtoPC: std_logic_vector(31 downto 0); -- MUX to PC

-- others
signal IMoutput: std_logic_vector(31 downto 0); -- InstructionMemory output
signal MUXtoReg: std_logic_vector(4 downto 0); -- MUX to RegisterFile
signal ExtendedSign: std_logic_vector(31 downto 0); -- SignExtension to Adder (Shifter
doesnt exist)

signal RegOut1: std_logic_vector(31 downto 0); -- RegisterFile to ALU
signal RegOut2: std_logic_vector(31 downto 0); -- RegisterFile to MUX
signal MUXtoALU: std_logic_vector(31 downto 0); -- MUX to ALU
signal ALUcontrolUnitToALU: std_logic_vector(3 downto 0);-- ALUcontrolUnit to ALU
signal ALUout: std_logic_vector(31 downto 0); -- ALU output towards DataMemory or MUX
signal DataWriteIn: std_logic_vector(31 downto 0); -- RegisterFile output towards
DataMemory
signal DMout: std_logic_vector(31 downto 0); --DataMemory to MUX


begin

          number1 <= std_logic_vector(to_unsigned(1, 32));
          FA: FullAdder port map (FAin1 => PCtoFAorIM, FAin2 => number1, Cin => "0",
Sum => FAplusplusOut); --Cin is String (0 downto 0)

          FA2: FullAdder port map(FAin1 => FAplusplusOut, FAin2 => ExtendedSign, Cin =>
"0", Sum => FA2toMUX); --Cin is String (0 downt 0)

          PC: ProgramCounter port map(PCin => MUXtoPC, PCout => PCtoFAorIM, clock =>
clock, reset => reset);

          IM: InstructionMemory port map(IMin => PCtoFAorIM, IMout => IMoutput);

          MUX_RF: MUX5 port map(MUXin1 => IMoutput(20 downto 16), MUXin2 => IMoutput(15
downto 11), MUXout => MUXtoReg, en => RegDst);

          RF: RegisterFile port map(clock => clock, RegIn1 => IMoutput(25 downto 21),
RegIn2 => IMoutput(20 downto 16), RegWriteIn => MUXtoReg, DataWriteIn => DataWriteIn,
```

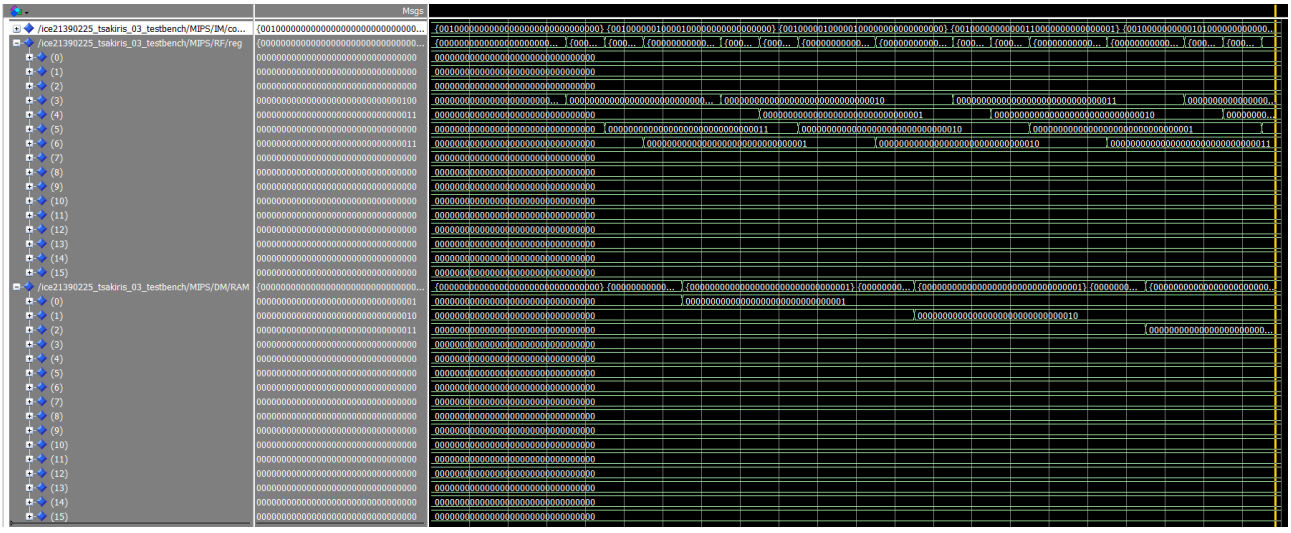
```
RegWrite => RegWrite, RegOut1 => RegOut1, RegOut2 => RegOut2);

          CU: ControlUnit port map(clock => clock, OPcode => IMoutput(31 downto 26),
RegWrite => RegWrite, ALUsrc => ALUSrc, ALUop => ALUop, MemWrite => MemWrite, MemRead
=> MemRead, RegDst => RegDst, MemToReg => MemToReg, Branch => Branch);

          ALU_CU: ALUcontrolUnit port map(ALUop => ALUop, Funct => IMoutput(5 downto
0), ALUcontrolUnitOut => ALUcontrolUnitToALU);

          S_EXT: SignExtender port map(SignIn => IMoutput(15 downto 0), SignOut =>
ExtendedSign);

          MUX_ALU: MUX32 port map(MUXin1 => RegOut2, MUXin2 => ExtendedSign, MUXout =>
MUXtoALU, en => ALUSrc);

          A_LU: ALU port map(ALUin1 => RegOut1, ALUin2 => MUXtoALU, op =>
ALUcontrolUnitToALU, ALUout => ALUout, Zero => Zero);

          DM: DataMemory port map(clock => clock, DMin => ALUout, WriteData => RegOut2,
MemWrite => MemWrite, MemRead => MemRead, DMout => DMout, reset => reset);

          MUX_DM: MUX32 port map(MUXin1 => ALUout, MUXin2 => DMout, MUXout =>
DataWriteIn, en => MemToReg);

          BRNCH: ANDgate port map(ANDin1 => Branch, ANDin2 => Zero, ANDout =>
BranchOut);

          MUX_BRNCH: MUX32 port map(MUXin1 => FAplusplusOut, MUXin2 => FA2toMUX, MUXout
=> MUXtoPC, en => BranchOut);

end structural;
```

Testbench

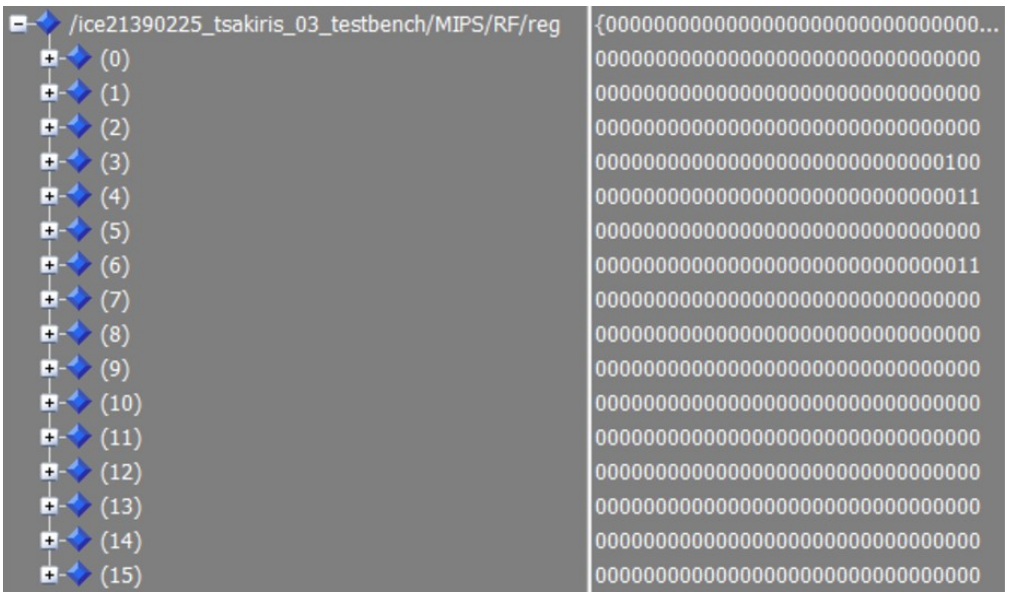
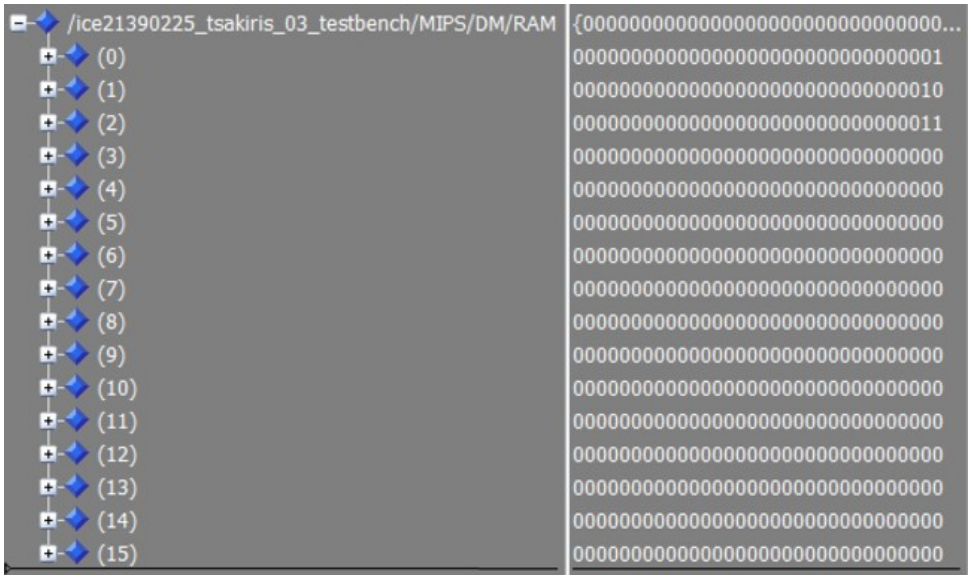
```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity MIPS_testbench is
end MIPS_testbench;

architecture testbench of MIPS_testbench is

          component MIPS port(
                    reset: in std_logic;
                    clock: in std_logic);
          end component;

          -- Constants
    constant CLOCK_PERIOD: time := 50 ns;

    -- Signals
    signal clock: std_logic := '0';
    signal reset: std_logic := '0';
begin

    -- Instantiate the components
          MIPS: MIPS port map(reset => reset, clock => clock);

    -- Clock process
    CLK_process :process begin
                    if now < CLOCK_PERIOD * 22 then -- 19 clock periods are enough to
execute all the instructions
                              clock <= '0';
                              wait for CLOCK_PERIOD/2;
                              clock <= '1';
                              wait for CLOCK_PERIOD/2;
                    else
                              wait;
                    end if;
   end process;


   -- Stimulus process
   stim_proc: process begin
                    reset <= '1';
                    wait for CLOCK_PERIOD/2;
                    reset <= '0';
                    wait;
   end process;

end testbench;
```

Ενδεικτικό τρέξιμο

Screenshot των καταχωρητών και της μνήμης από την εκτέλεση:



Screenshot μόνο των τιμών των καταχωρητών:



Screenshot μόνο των τιμών των θέσεων μνήμης: